

Sicurezza nelle Comunicazioni (anche App di Messaggistica)

NECKER

5 Dicembre 2025

Contents

1	MAC	2
1.1	MAC: Media Access Control Address (Networking)	2
1.2	MAC: Message Authentication Code (Sicurezza)	2
2	Principio di Funzionamento del MAC	3
2.1	Generazione del Tag: $t = M_k(m)$ (Lato Mittente)	3
2.2	Verifica del Tag: $t'' \stackrel{?}{=} t'$ (Lato Destinatario)	3
3	Crittografia e Livelli del Modello OSI	5
4	Protocolli di Sicurezza: IPsec, HTTPS e TLS	7
4.1	IPsec (Internet Protocol Security) - Livello 3 (Rete)	7
4.2	TLS (Transport Layer Security) - Livello 4/5 (Trasporto)	8
4.3	HTTPS (Hypertext Transfer Protocol Secure) - Livello 7 (Applicazione)	9
5	Cifratura End-to-End e Scambio Chiavi	10
5.1	Cifratura End-to-End (E2EE)	10
5.2	Algoritmo di Scambio Chiavi Diffie-Hellman (DH)	10
6	Protocolli Moderni per la Messaggistica Sicura	12
6.1	1. MTProto (Mobile Telegram Protocol)	12
6.2	2. Signal Protocol (Ex-TextSecure)	12
7	Analisi Comparativa delle Piattaforme di Messaggistica	13
7.1	Protocolli e Cifratura (Focus Riservatezza)	13
7.2	Riservatezza e Metadati (Focus Privacy)	13
7.3	Analisi della Riservatezza: Limiti Pratici dell'E2EE e Backdoor Funzionali	13

1 MAC

L'acronimo MAC identifica due concetti fondamentali ma totalmente distinti nell'informatica e nella crittografia: il **Media Access Control Address** e il **Message Authentication Code**.

1.1 MAC: Media Access Control Address (Networking)

Il MAC Address è un **identificatore hardware** che opera a livello locale all'interno delle reti.

- **Natura:** È un **indirizzo fisico** permanente e unico, codificato nella scheda di rete (NIC) dal produttore.
- **Funzione:** Identificare in modo univoco un dispositivo all'interno di una rete locale (LAN), permettendo ai pacchetti dati di raggiungere il dispositivo corretto (livello 2 del modello OSI).
- **Formato:** È una sequenza fissa di 48 bit (6 byte), rappresentata in formato esadecimale (es. 00:1A:2B:3C:4D:5E).
- **Livello di Rete:** Opera al livello Data Link (livello 2 del modello OSI).
- **Esempio di utilizzo:** Utilizzato dai protocolli ARP (Address Resolution Protocol) e Ethernet.

1.2 MAC: Message Authentication Code (Sicurezza)

Il MAC in ambito crittografico è un **meccanismo di sicurezza** orientato alla verifica dei dati in transito. [\(trattato in questo documento\)](#)

- **Natura:** È un **codice crittografico** dinamico, generato per ogni messaggio.
- **Funzione:** Garantire l'**integrità** (che il messaggio non sia stato alterato) e l'**autenticità** (che provenga da chi possiede la chiave segreta) del messaggio **ma non la segretezza**.
- **Input e Output:** La sua generazione richiede il messaggio (m) e una **chiave segreta simmetrica** (k), producendo un tag (t).

$$t = M_k(m)$$

- **Livello di Rete:** Opera a livello applicativo o di trasporto (per esempio in TLS/SSL).
- **Esempio di utilizzo:** HMAC (Keyed-Hash Message Authentication Code), CMAC.

2 Principio di Funzionamento del MAC

Il **Message Authentication Code (MAC)** è una funzione crittografica che garantisce l'**integrità** e l'**autenticità** di un messaggio m utilizzando una chiave segreta simmetrica k . L'algoritmo MAC viene indicato con M .

2.1 Generazione del Tag: $t = M_k(m)$ (Lato Mittente)

La fase di generazione del Tag MAC è eseguita dal mittente e ha l'obiettivo di creare un codice di autenticazione univoco per il messaggio.

1. **Input:** Il mittente prende in input il **messaggio originale** (m') di lunghezza arbitraria e la **chiave segreta simmetrica** (k), che deve essere nota solo al mittente e al destinatario.
2. **Elaborazione:** L'algoritmo MAC (M) viene applicato a m' e k . Il MAC non è una semplice funzione hash, ma una **funzione crittografica** che dipende dalla chiave.
3. **Output:** Il risultato dell'elaborazione è il **Tag** (t'), una sequenza di bit di lunghezza fissa (es. 128 bit), calcolata come:

$$t' = M_k(m')$$

4. **Trasmissione:** Il mittente invia al destinatario la coppia composta dal **messaggio originale** (m') e dal **Tag** (t').

L'inviolabilità del MAC garantisce che un attaccante non in possesso di k non possa forgiare un tag corretto per un messaggio modificato.

2.2 Verifica del Tag: $t'' \stackrel{?}{=} t'$ (Lato Destinatario)

La fase di verifica è eseguita dal destinatario per confermare l'integrità e l'autenticità del messaggio ricevuto.

1. **Ricezione:** Il destinatario riceve il messaggio potenziale (m') e il tag ricevuto (t').
2. **Ricalcolo del Tag:** Utilizzando la **stessa chiave segreta** (k) e lo **stesso algoritmo MAC** (M), il destinatario ricalcola il tag sul messaggio ricevuto (m'), ottenendo il tag di verifica (t''):

$$t'' = M_k(m')$$

3. **Confronto:** Il destinatario confronta rigorosamente il tag calcolato (t'') con il tag ricevuto (t').

$$\text{Valutazione: } t'' \stackrel{?}{=} t'$$

4. **Esito:**

- Se $t'' = t'$, il messaggio è **accettato**. L'integrità è confermata (il messaggio non è stato alterato) e l'autenticità è garantita (è stato generato da chi possiede k).
- Se $t'' \neq t'$, il messaggio è **rifiutato**. Indica che il messaggio è stato modificato (violazione dell'integrità) o è stato inviato da una fonte non autorizzata (violazione dell'autenticità).

Diagramma del Flusso MAC

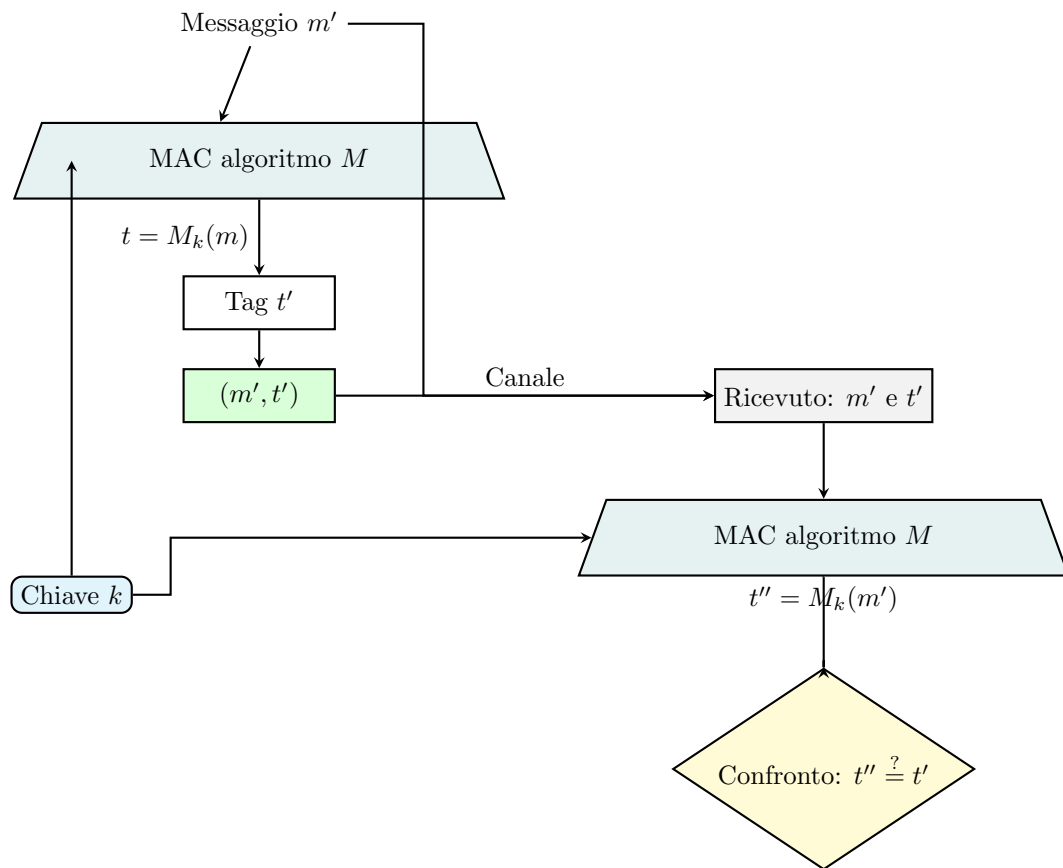


Figure 1: Flusso di Generazione e Verifica del Message Authentication Code (MAC).

3 Crittografia e Livelli del Modello OSI

Introdotta il **MAC**, per garantire la **riservatezza** (confidentiality) introduciamo la crittografia, la quale può avvenire a diversi livelli del modello OSI (Open Systems Interconnection). La scelta del livello determina la **portata della protezione** e quali **intestazioni** (header) del pacchetto vengono cifrate.

Livello 7: Applicazione (Application Layer)

Questo è il livello in cui la crittografia è applicata direttamente al dato **utile all'utente** prima che venga consegnato agli strati inferiori.

- **Cosa viene cifrato:** Solo il contenuto del messaggio o il corpo del file. Le intestazioni dell'applicazione (es. l'oggetto di una email) possono restare in chiaro.
- **Vantaggi:** La riservatezza è mantenuta **end-to-end**, anche se i dati passano attraverso nodi intermedi non fidati. Ideale per la posta elettronica (PGP, S/MIME).
- **Esempi:** PGP (Pretty Good Privacy) o GnuPG per le email.

Livello 6/5: Presentazione/Sessione (Presentation/Session Layer)

Spesso questi livelli vengono gestiti in combinazione per la crittografia di sessione.

- **Cosa viene cifrato:** I dati dell'applicazione e in parte alcune informazioni di sessione/presentazione.
- **Vantaggi:** Protegge l'intera sessione di comunicazione. È trasparente alle applicazioni sopra e alle reti sotto.
- **Esempi:** SSL/TLS (Secure Sockets Layer/Transport Layer Security) non strettamente a Livello 6, ma la sua implementazione logica è spesso posizionata qui o al di sopra del Livello 4.

Livello 4: Trasporto (Transport Layer)

Questo livello protegge la comunicazione tra i **due endpoint** (client e server).

- **Cosa viene cifrato:** L'intero segmento di dati (payload) inviato tra due host. Le intestazioni di rete (IP) e di trasporto (TCP/UDP) restano in chiaro.
- **Vantaggi:** Fornisce riservatezza e integrità tra le due applicazioni terminali. Il router intermedio non può leggere i dati ma può instradarli.
- **Esempi:** TLS/SSL (la sua implementazione pratica è spesso vista qui), SSH (Secure Shell).

Livello 3: Rete (Network Layer)

Questo è il livello di crittografia che protegge il **datagramma IP** (tranne le informazioni di instradamento).

- **Cosa viene cifrato:** L'intero pacchetto dati (incluso il payload e le intestazioni di Livello 4), **tranne le intestazioni IP esterne** (necessarie per l'instradamento).
- **Vantaggi:** Protezione su base **hop-by-hop** o **tunneling** tra due gateway di rete. È trasparente per l'utente e per le applicazioni.
- **Esempi:** IPsec (Internet Protocol Security), che può operare in modalità trasporto o tunnel.

Livello 2: Data Link (Data Link Layer)

La crittografia qui si concentra sulla protezione del traffico tra due nodi collegati direttamente (es. un PC e un router o due nodi di una LAN).

- **Cosa viene cifrato: Tutto il frame** (pacchetto dati completo, incluse intestazioni IP e TCP), **tranne l'intestazione fisica** (necessaria al livello 1 per la trasmissione).
- **Vantaggi:** Protezione su base **hop-by-hop** (tra due dispositivi collegati fisicamente).
- **Esempi:** **MACsec** (Media Access Control Security), **WPA2/3** per reti Wi-Fi (che operano a cavallo tra Livello 2 e 1).

4 Protocolli di Sicurezza: IPsec, HTTPS e TLS

I protocolli di sicurezza operano a diversi livelli della pila di rete per garantire riservatezza, integrità e autenticità. IPsec opera al Livello di Rete, mentre HTTPS si basa su TLS, che opera tra i Livelli di Trasporto e Applicazione.

4.1 IPsec (Internet Protocol Security) - Livello 3 (Rete)

IPsec è una **suite di protocolli standardizzata** che estende il Protocollo Internet (IP) di Livello 3, aggiungendo funzionalità di sicurezza essenziali direttamente al livello di rete. La sua forza risiede nella capacità di fornire protezione **trasparente** a tutti i protocolli e applicazioni che si trovano al di sopra dello strato IP.

Componenti e Funzioni

IPsec si basa su due protocolli chiave per definire i servizi di sicurezza:

1. AH (Authentication Header) - Integrità e Autenticità:

- **Scopo:** Garantisce l'**integrità** (verifica che il pacchetto non sia stato modificato) e l'**autenticità** (verifica l'origine del pacchetto) del pacchetto IP.
- **Meccanismo:** Calcola un **MAC** (Message Authentication Code) sul payload e sulla maggior parte dell'intestazione IP. L'intestazione IP mutabile (es. TTL) è esclusa dal calcolo.
- **Riservatezza:** AH **non offre cifratura**, quindi il contenuto (payload) del pacchetto IP rimane in chiaro.

2. ESP (Encapsulating Security Payload) - Riservatezza, Integrità e Autenticità:

- **Scopo:** È il protocollo più versatile e comune. Fornisce **riservatezza** (cifratura) e opzionalmente **integrità** e **autenticità**.
- **Meccanismo:** Cifra il **payload** e incapsula il tutto, aggiungendo un'intestazione ESP prima del payload e un trailer ESP alla fine.
- **Uso Combinato:** Nella pratica, ESP è quasi sempre configurato per offrire sia cifratura che autenticazione, rendendolo superiore ad AH per la protezione completa dei dati.

Modalità Operative (Configurazioni)

IPsec può essere configurato per operare in due modalità distinte, che differiscono per ciò che viene protetto e per come il pacchetto viene incapsulato:

• Modalità Trasporto (Transport Mode):

- **Protezione:** Solo il **payload** (dati di Livello 4 e superiori) del pacchetto IP viene protetto (cifrato e/o autenticato).
- **Intestazione IP:** L'intestazione IP originale rimane **in chiaro** e viene usata per l'instradamento.
- **Utilizzo:** Usato per la protezione **end-to-end** (host-to-host) quando i due endpoint sono IPsec-compatibili e si trovano sulla stessa rete o quando si vuole nascondere solo il payload applicativo.

• Modalità Tunnel (Tunnel Mode):

- **Protezione:** L'intero **pacchetto IP originale** (intestazione + payload) viene protetto (cifrato e/o autenticato).
- **Incapsulamento:** Il pacchetto protetto viene **incapsulato** all'interno di una **nuova intestazione IP esterna**.
- **Utilizzo:** Base fondamentale delle **VPN** (Virtual Private Network). Viene utilizzato per creare un tunnel protetto tra due gateway di rete (site-to-site) o tra un host e un gateway (remote-access). Nasconde l'indirizzo IP interno (originale) nel pacchetto cifrato impedendo l'identificazione dei due utenti che comunicano.

4.2 TLS (Transport Layer Security) - Livello 4/5 (Trasporto)

- **Definizione:** TLS è il protocollo crittografico successore di SSL (Secure Sockets Layer), progettato per garantire la sicurezza delle comunicazioni su reti di computer.
- **Funzione:** Fornisce riservatezza (cifratura), integrità (MAC) e autenticazione tra due applicazioni (es. browser e web server) che comunicano sopra il protocollo TCP.
- **Fasi Principali (Handshake):**
 1. **Hello e Negoziazione (Inizio Sessione):**
 - **Client Hello:** Il client avvia la comunicazione inviando un messaggio **Client Hello** che include le versioni TLS supportate, un elenco degli algoritmi crittografici preferiti (Cipher Suites) e un **numero casuale client** (Nonce).
 - **Server Hello:** Il server risponde con un messaggio **Server Hello** selezionando la versione TLS e la Cipher Suite che utilizzerà, e invia il proprio **numero casuale server**.
 2. **Autenticazione e Scambio Chiavi:**
 - **Certificato e Scambio Chiavi Asimmetrico:** Il server invia il suo **Certificato Digitale** (contenente la chiave pubblica) al client, che lo verifica. Successivamente, i due endpoint utilizzano protocolli di scambio chiavi (es. Diffie-Hellman) e la crittografia asimmetrica per calcolare la **Chiave Simmetrica di Sessione** (Premaster Secret), senza che questa venga mai trasmessa in chiaro.
 - **Finished:** Entrambe le parti inviano un messaggio cifrato per confermare che l'handshake è terminato con successo e che sono pronte per la comunicazione sicura.
 3. **Trasmissione Dati (Record Protocol):**
 - Una volta stabilita la sessione, i dati di applicazione vengono suddivisi in record, che vengono **cifrati** (riservatezza) e autenticati tramite un **MAC** (integrità) utilizzando la chiave simmetrica concordata.
- **Livello di Rete:** Opera a cavallo tra il **Livello Applicazione** (per l'handshake e l'uso dei certificati) e il **Livello Trasporto** (per l'incapsulamento sopra TCP).
- **Vantaggi:** Ampiamente supportato, essenziale per il web, fornisce protezione **end-to-end** a livello di sessione.

Cipher Suites: Algoritmi e Restrizioni Storiche

1. Spiegazione e Struttura della Cipher Suite

Una **Cipher Suite** (o Cyphersuite) è un insieme di algoritmi crittografici concordati durante l'Handshake TLS/SSL che definiscono **come** avverrà la comunicazione sicura. Ogni suite è composta da quattro elementi chiave:

Componente	Funzione e Esempio (Esempio: TLS_RSA_WITH_AES_128_GCM_SHA256)
Protocollo	Indica il protocollo di sicurezza (es. SSL o TLS).
Algoritmo di Scambio Chiavi	Metodo per scambiare la chiave simmetrica di sessione (es. RSA , Diffie-Hellman, ECDHE).
Algoritmo di Cifratura	Algoritmo simmetrico usato per cifrare i dati (es. RC4, DES, AES_128_GCM).
Algoritmo Hash/MAC	Funzione hash usata per l'integrità e l'autenticazione (es. MD5, SHA, SHA256).

Figure 2: Struttura di una Cipher Suite TLS/SSL.

Le Cipher Suites e le Restrizioni all'Esportazione

Le Cipher Suites che includono EXPORT nel loro nome (SSL_RSA_EXPORT_WITH...) sono un retaggio storico delle **guerre della crittografia** negli Stati Uniti e dei trattati post-bellici sul controllo delle armi.

Implicazione: Impossibilità di Comunicare Inizialmente

Il problema di negoziare una suite EXPORT non era l'impossibilità totale di comunicare, ma la **necessità di degradare intenzionalmente la sicurezza** all'inizio della comunicazione per rispettare le leggi di esportazione.

Durante l'Handshake TLS/SSL (fase di negoziazione), se un client fuori dagli USA si connetteva a un server USA, l'unica **Cipher Suite legale comune** che potevano concordare era una debole suite EXPORT. Questo rendeva la connessione vulnerabile all'intercettazione da parte di agenzie governative o attaccanti esterni.

4.3 HTTPS (Hypertext Transfer Protocol Secure) - Livello 7 (Applicazione)

- **Definizione:** HTTPS **non** è un protocollo di sicurezza a sé stante, ma è semplicemente l'uso del protocollo HTTP (Hypertext Transfer Protocol) **sovrapposto** a uno strato di sicurezza **TLS/SSL**.
- **Formula:** La relazione è definita come:

$$\text{HTTPS} = \text{HTTP} + \text{TLS/SSL}$$

- **Porta:** Utilizza la porta standard **443**, invece della porta 80 di HTTP.
- **Funzione:** Tutti i dati scambiati tra il browser (client) e il web server, inclusi URL, intestazioni e contenuto dei form, vengono cifrati e autenticati dal sottostante strato TLS.
- **Obiettivo:** Protezione delle **transazioni web** e dell'identità dell'utente, garantendo che l'utente stia parlando con il server corretto.

5 Cifratura End-to-End e Scambio Chiavi

Queste sezioni illustrano due concetti fondamentali della crittografia moderna: garantire che solo i partecipanti finali leggano i dati (**E2EE**) e come stabilire un canale segreto per la comunicazione (**Diffie-Hellman**).

5.1 Cifratura End-to-End (E2EE)

La Cifratura End-to-End (E2EE) è un sistema di comunicazione in cui **soltanto gli utenti che comunicano possono leggere i messaggi**. Nessuna terza parte, inclusi i provider di servizi Internet, i gestori delle reti, o la società che offre il servizio di comunicazione (es. la piattaforma di messaggistica), ha la possibilità di accedere al contenuto in chiaro dei messaggi o dei file scambiati.

- **Principio Cardine:** La cifratura e la decifratura avvengono esclusivamente sui **dispositivi degli utenti finali**. Il messaggio viene cifrato dal mittente e decifrato solo dal destinatario.
- **Stato Intermedio:** Tutti i messaggi che viaggiano tra i due endpoint (e che sono immagazzinati sui server intermedi) rimangono in **formato cifrato (illeggibile)**.
- **Chiavi d'Accesso:** I provider di servizi **non possiedono le chiavi private** necessarie per decifrare i messaggi. Questo contrasta con la crittografia **hop-by-hop** (o transport-layer encryption, come il TLS tra browser e server), dove il messaggio viene decifrato sul server prima di essere eventualmente ricifrato per la destinazione.

5.2 Algoritmo di Scambio Chiavi Diffie-Hellman (DH)

L'algoritmo Diffie-Hellman è un metodo crittografico fondamentale inventato nel 1976 che consente a due parti che non hanno mai interagito tra loro e che comunicano su un canale **non sicuro** di stabilire congiuntamente una **chiave segreta simmetrica** condivisa.

Principio di Base: Funzione Unidirezionale e Problema del Logaritmo Discreto

L'efficacia di DH si basa su una proprietà matematica nota come il **Problema del Logaritmo Discreto**. È facile eseguire l'elevamento a potenza (la direzione "avanti"), ma è estremamente difficile e computazionalmente oneroso eseguire l'operazione inversa (il logaritmo discreto) per risalire all'esponente originale.

Fasi dell'Algoritmo (Scambio del "Colore")

Assumiamo che Alice e Bob vogliano concordare un colore segreto comune, partendo da un colore pubblico e noto a tutti (la base).

1. **Scelta Pubblica (Base):** Alice e Bob concordano su un grande **numero primo pubblico** (p) e una **radice primitiva pubblica** (g). Questi valori sono noti a chiunque, incluso un intercettore.
2. **Selezione Segreta:**
 - Alice sceglie un **numero segreto casuale** (a).
 - Bob sceglie un **numero segreto casuale** (b).
3. **Scambio dei Valori Pubblici Parziali:**
 - Alice calcola e invia a Bob il valore A :

$$A = g^a$$

- Bob calcola e invia ad Alice il valore B :

$$B = g^b$$

(intercettore intercetta A e B , ma non conosce a o b).

4. Calcolo della Chiave Segreta Condivisa (K):

- Alice utilizza il valore di Bob (B) con la sua chiave segreta (a):

$$K = B^a = (g^b)^a = g^{ab}$$

- Bob utilizza il valore di Alice (A) con la sua chiave segreta (b):

$$K = A^b = (g^a)^b = g^{ab}$$

Poiché g^{ab} è identico per entrambi, **Alice e Bob hanno ottenuto la stessa chiave segreta K** senza che Eve sia riuscita a determinarla (perché dovrebbe risolvere il logaritmo discreto per trovare a o b da A o B). La chiave K viene quindi utilizzata per la **cifatura simmetrica** ad alta velocità dei dati della sessione.

6 Protocolli Moderni per la Messaggistica Sicura

6.1 1. MTPROTO (Mobile Telegram Protocol)

MTPROTO è il protocollo crittografico proprietario sviluppato specificamente per l'applicazione di messaggistica **Telegram**. È progettato per essere ottimizzato per le connessioni mobili e per gestire grandi quantità di dati con efficienza.

- **Struttura:** MTPROTO è diviso in tre strati:

1. **Strato di Alto Livello (API):** Definisce i metodi e gli oggetti per le query API e la comunicazione tra client e server.
2. **Strato Crittografico:** Gestisce l'autorizzazione, l'autenticazione tramite chiavi RSA e lo scambio di chiavi Diffie-Hellman per stabilire una chiave di sessione AES.
3. **Strato di Trasporto:** Fornisce il trasporto dei dati (spesso su TCP o HTTP).

- **Modalità di Cifratura:**

- **Chat Normali/Gruppi (Cloud Chat):** I messaggi sono cifrati **client-server/server-client**. I dati sono cifrati tra il tuo dispositivo e il server Telegram, ma **risiedono cifrati sui server** (consentendo la sincronizzazione su più dispositivi). Questo non è End-to-End.
- **Chat Segrete (Secret Chat):** Utilizzano la cifratura **End-to-End (E2EE)**. I messaggi non vengono memorizzati sul server e sono accessibili solo sui dispositivi partecipanti. Supportano l'autodistruzione e sono legate a un singolo dispositivo.

ATTENZIONE: vedremo che in realtà neanche questa è **End-to-End**

- **Critiche:** Essendo un protocollo proprietario e non standardizzato, ha ricevuto critiche per la sua complessità e per la mancanza di una verifica approfondita da parte della comunità crittografica esterna, sebbene non siano state trovate vulnerabilità critiche.

6.2 2. Signal Protocol (Ex-TextSecure)

Signal Protocol (sviluppato dalla Open Whisper Systems e ora utilizzato dall'app Signal) è ampiamente considerato il protocollo **gold standard** per la sicurezza delle comunicazioni e la cifratura End-to-End (E2EE).

- **Struttura e Scopo:** Progettato per essere asincrono (funziona anche offline) e per fornire E2EE forte per messaggi, chiamate vocali e video.
- **Tecnologie Chiave:**
 - **Double Ratchet Algorithm (DMA):** Utilizza una combinazione di algoritmi Diffie-Hellman e funzioni hash per generare continuamente nuove chiavi simmetriche per **ogni singolo messaggio** (**MTPROTO ogni 100 messaggi**).
 - **Forward Secrecy (Segretezza in Avanti):** Se una chiave di sessione viene compromessa, solo i messaggi futuri, poiché le chiavi passate vengono eliminate (in avanti poiché andando avanti i messaggi indietro sono protetti).
 - **Future Secrecy (Segretezza in Indietro):** Se la chiave corrente viene compromessa, le chiavi future generate sono protette.
- **Implementazione:** Il protocollo è **open source** ed è soggetto a continua revisione da parte di esperti crittografi, motivo per cui è adottato da molte altre piattaforme come WhatsApp, Facebook Messenger e Google Messages.

7 Analisi Comparativa delle Piattaforme di Messaggistica

La sicurezza di un'applicazione non dipende solo dal protocollo, ma anche dalla sua implementazione, dalle impostazioni predefinite e dalle politiche sulla raccolta dei metadati.

7.1 Protocolli e Cifratura (Focus Riservatezza)

App	Protocollo di Cifratura	E2EE Predefinito
Signal	Signal Protocol	Sì (1:1 e Gruppi)
WhatsApp	Signal Protocol	Sì (1:1 e Gruppi)
Telegram	MTPProto	No (Solo Chat Segrete)
iMessage	Proprietario Apple	Sì (tra utenti Apple)
Snapchat	TLS + Proprietario	Parziale (solo chiamate/video)
Instagram (DM)	Trasporto (TLS)	No E2EE
TikTok (DM)	Trasporto (TLS)	No E2EE

7.2 Riservatezza e Metadati (Focus Privacy)

App	Livello di Riservatezza	Raccolta Metadati / Info Rivelate
Signal	Massima	Minima (Solo numero di telefono e data ultimo accesso).
WhatsApp	Ottima	Alta (IP, contatti, ID dispositivo, chi parli. Utilizzati da Meta per l'analisi).
Telegram	Variabile	Alta (IP, Contatti, ID dispositivo, sincronizzazione rubrica. Necessari per la funzionalità Cloud).
iMessage	Buona	Media (Metadati e backup gestiti da Apple).
Snapchat	Media	Alta (Posizione, contatti, interazioni, dati demografici per advertising).
Instagram (DM)	Bassa	Massima (Interazioni, interessi, dati per advertising, legati all'ecosistema Meta).
TikTok (DM)	Bassa	Massima (Interessi, dati di visualizzazione, dati di rete usati per l'algoritmo).

7.3 Analisi della Riservatezza: Limiti Pratici dell'E2EE e Backdoor Funzionali

La sicurezza offerta dalla crittografia End-to-End (E2EE) è teoricamente robusta, ma la sua effettiva **riservatezza (privacy)** è spesso compromessa da funzionalità di comodità e scelte di design che creano potenziali punti di accesso per il provider.

1. Impostazione Predefinita E2EE e Funzionalità Cloud:

- **Telegram (Cloud Chat):** È fondamentale ribadire che Telegram per le chat standard (Cloud Chat) utilizza la cifratura Client-Server, e non E2EE. I messaggi sono archiviati sui server. Per le **Chat Segrete** (E2EE), Telegram utilizza la sua implementazione di scambio chiavi. **(Middle, dipende dall'implementazione)**

2. Derivazioni e Punti di Compromissione in WhatsApp (Meta):

Nonostante l'uso del robusto Signal Protocol, WhatsApp presenta eccezioni che riducono la portata assoluta dell'E2EE:

- **Backup Cloud (Decifrazione Esterna):** Per la comodità dell'utente, i backup su Google Drive o iCloud **non sono cifrati End-to-End per default** (anche se WhatsApp ha introdotto l'opzione di cifratura E2EE per i backup stessi). Se l'utente non attiva questa opzione, le copie dei messaggi vengono decifrate e cifrate con chiavi gestite dai provider cloud (Google/Apple), rendendole potenzialmente accessibili a terzi. **(non sicuro)**

- **Accesso Web/Desktop (Decifrazione su Dispositivo):** Funzionalità come WhatsApp Web o l'app Desktop richiedono che i messaggi vengano **decifrati sul dispositivo mobile principale** e poi **trasmessi attraverso un canale sicuro separato** al client secondario. (**sicuro**)
- **Anteprime dei Messaggi (Metadati Decifrati):** Per mostrare l'anteprima dei link condivisi (ad esempio, il titolo di un articolo), WhatsApp deve inviare l'URL del link ai **server di Meta** (o un servizio di caching) affinché questi ne traggano le informazioni necessarie (titolo, immagine). Sebbene il testo del messaggio rimanga cifrato, l'URL (che è un metadato sensibile che rivela il contenuto della conversazione) viene **temporaneamente decifrato o esposto ai server**. (**mid, solo i link condivisi**)

3. Raccolta dei Metadati (Ciò che Sanno di Noi):

- **Metadati (Tutte le App):** I metadati (chi comunica, quando, da dove, con quale dispositivo) **non sono mai cifrati End-to-End**. Aziende come **Meta** (WhatsApp) e **ByteDance** (TikTok) raccolgono metadati estremamente dettagliati sul comportamento e sulle relazioni degli utenti, utilizzandoli per il targeting pubblicitario e l'analisi del grafico sociale, anche se non possono leggere il contenuto dei messaggi E2EE. (**non sicuro**)
- **Signal (Minima Raccolta):** Raccoglie la quantità minima di metadati per il funzionamento (solo numero di telefono e data dell'ultimo accesso), limitando drasticamente la superficie di attacco alla privacy. (**sicuro**)

La **riservatezza assoluta** richiede non solo un protocollo E2EE forte, ma anche l'eliminazione di funzionalità come backup cloud non cifrati, accesso multi-dispositivo non rigoroso e l'assenza totale di analisi dei metadati.